

SENIOR REACT DEVELOPER

Frontend Technical Assessment

AI-Assisted Engineering Task

Expected active work: 4 hours | **Submission window: 48 hours** | **AI tools: Allowed**

Build a small, production-quality frontend application using React and TypeScript. The task is intentionally open-ended in architecture so you can demonstrate engineering judgment.

Important: You may use AI coding tools. We care about your ability to make engineering decisions, review generated code, debug it, and take ownership of the final system.

1. Objective

Build a Developer Intelligence Dashboard that allows a user to search GitHub developers and repositories and explore repository details.

You should build this as you would a small real product: clean architecture, sensible UX, reliable API handling, strong TypeScript, and thoughtful handling of edge cases.

No backend, private infrastructure, database, or company credentials are required.

2. Time & Submission Rules

Active Development Time

Maximum active development time: 4 hours. Please time-box your work. Do not turn this into a multi-day project.

Submission Window

Submit within 48 hours of receiving this assessment. You may choose when to work within that window; the expected active effort remains approximately four hours.

If You Run Out of Time

Stop at the four-hour active-work limit and document what remains unfinished. Prioritization is part of the task.

Do Not Over-Polish

Prioritize correctness, architecture, reliability, and usability over animations or excessive visual polish.

3. Technology

- React
- TypeScript
- A modern frontend build tool such as Vite
- Any reasonable styling approach
- Any reasonable state-management approach

You are free to choose libraries and tools. We are intentionally not prescribing Redux, Zustand, React Query, Tailwind, a UI kit, or a particular folder structure.

4. Public API

Use the public GitHub REST API:

<https://api.github.com/>

No API key is required for the basic assignment.

```
GET /search/repositories?q={query}
GET /search/users?q={query}
GET /users/{username}
GET /repos/{owner}/{repo}
GET /repos/{owner}/{repo}/issues
```

You may use additional public APIs if you believe they meaningfully improve the product, but this is optional.

5. Required Features

5.1 Search

- Search GitHub repositories.
- Search GitHub users.
- Allow the user to switch between repository and user search.
- Display useful search results.
- Implement pagination or another sensible result-loading approach.

5.2 Repository Results

- Repository name
- Owner and avatar
- Description
- Primary language
- Stars
- Forks
- Open issues
- Last updated date
- GitHub URL

5.3 User Results

- Avatar
- Username
- Name where available
- Followers
- Following
- Public repositories
- Location where available
- GitHub profile URL

5.4 Repository Details

- Repository name and description
- Owner
- Stars, forks and watchers
- Open issues
- Language
- Created and updated dates
- Default branch
- GitHub URL
- Recent repository issues

5.5 Issues

Display recent issues with title, state, author, created/updated date, issue number, and GitHub URL.

6. UX & Reliability Requirements

Loading States

Every API-driven area needs a clear loading experience. A simple text loader is acceptable, but use your judgment about where skeletons or progressive loading improve the experience.

Empty States

Handle no search results, repositories with no issues, and other empty states clearly.

Error States

- Network failure
- GitHub API failure
- Rate limiting
- Invalid search/request
- Failed repository detail request
- Failed issue request

Do not leave the user with a blank or broken screen after an API failure. Provide retry behavior where appropriate.

Responsive Design

The application should work well on desktop, tablet, and mobile.

Accessibility

Use semantic HTML, accessible labels, sensible keyboard behavior, focus handling where relevant, and reasonable contrast.

7. Architecture & Engineering Expectations

The implementation details are intentionally yours to decide. We want to see how you think about a real frontend system.

You should consider

- Component boundaries and responsibilities
- Local state versus shared state
- Server/API state
- API/service abstraction
- Type-safe API models
- Data transformation
- Error strategy
- Routing and URL state where useful
- Caching
- Request cancellation
- Search debouncing
- Race conditions
- Pagination
- Testing boundaries
- Performance

Important

Do not add complexity simply to demonstrate knowledge. Use the simplest architecture that appropriately solves the problem.

8. Required Engineering Decisions

Your README should explain the important decisions you made.

State Management

Explain what state is local, what is shared, how server state is handled, and why you chose the approach.

API Layer

Explain where API calls live, how API responses are typed, how errors are handled, and how API data is transformed.

Caching

Explain whether you use caching. If yes, describe what is cached and why. If no, explain why you decided it was unnecessary for this scope.

Search & Async Behavior

Consider debouncing, duplicate requests, stale responses, cancellation, pagination, URL state, and race conditions.

Performance

Consider repeated searches, large result sets, rendering cost, and network efficiency. Avoid unnecessary memoization.

9. AI Coding Tools Are Allowed

You may use AI freely during this assessment.

- Cursor
- Claude Code
- GitHub Copilot
- ChatGPT
- Gemini
- Windsurf
- Other AI coding assistants

You are not expected to manually type every line of code.

However, you are responsible for the final implementation. You should review generated code, reject incorrect suggestions, make architecture decisions yourself, and understand the code you submit.

AI Usage Disclosure

Add a short section to the README explaining:

- Which AI tools you used.
- What you used them for.
- What you designed or decided yourself.
- What generated code you reviewed or changed.
- Any important AI suggestion you rejected and why.

You do not need to provide a full prompt transcript.

10. README Requirements

Your repository must include a clear README containing:

- Project overview
- Setup and run instructions
- Architecture overview
- Key technical decisions
- Trade-offs
- Assumptions
- Known limitations
- Testing approach
- AI usage disclosure

For at least five important decisions, use this format:

```
Decision:  
Alternatives considered:  
Why I chose this:  
Trade-off:
```

11. Optional Enhancements

These are optional. Do not sacrifice core quality to implement them.

- Repository favorites
- Dark/light mode
- Repository comparison
- Language and minimum-star filters
- Keyboard navigation
- URL-persisted search state
- API caching
- Automated tests
- Improved accessibility
- Additional useful public API integration

12. Submission Requirements

Submit one Git repository.

- Complete source code
- README.md
- Working application
- Setup instructions
- AI usage disclosure

The application must run without private company infrastructure.

```
npm install
npm run dev
```

The reviewer should be able to clone the repository and run the application without requesting credentials or manual configuration from the hiring team.

13. What Happens After Submission

If selected for the next stage, you will have a short technical discussion based on your submitted project.

You may be asked to explain your architecture, API decisions, state management, performance considerations, and trade-offs.

You may also be asked to make a small change or debug a problem in your submitted application. AI tools may continue to be used during this discussion.

Example live change

Add repository filtering by language and minimum stars while preserving the existing search and pagination behavior.

Example debugging scenario

When users search rapidly, sometimes an older API response appears after the newer search result. Diagnose and fix the issue.

The purpose is to understand how you reason about and extend a system you built.

14. Final Guidance

There is no single correct architecture for this assignment.

We are interested in the quality of your decisions, not whether you selected a particular library.

A smaller application with strong engineering is preferable to a large application with unnecessary complexity.

Build like you are shipping a small feature in a real product.

Focus on: Architecture → Correctness → Reliability → UX → Maintainability → Performance → Polish.

Good luck.